Jane's structured approach—clarifying the question, breaking down specificity, and referencing real-world maintainability—impressed the interviewer. She emphasized that understanding specificity prevents unintended style conflicts, especially in React's component-driven architecture.

This example is located in `use-cases/CssSpecificityExample.tsx` and can be executed with the command: `VITE_NAME=CssSpecificityExample npm run dev`.

Jane further distinguished herself by discussing the specificity of newer CSS features like `:is()`, `:has()`, `:where()`, cascade layers (`@layer`), and scoped styles (`@scope`). These features offer advanced methods for organizing and controlling styles with greater precision over specificity. Traditionally, selectors like `#nav .btn:hover` were used, but now they can be replaced by more concise alternatives such as `:is(.header, #nav) .btn:where(:hover)`. This modern approach results in lower calculated specificity and helps reduce specificity wars by minimizing unintended style overrides.

She explained how the specificity of the selector `:is(.header, #nav) .btn:where(:hover)` is calculated: `:is()` takes the highest specificity among its arguments—here, `#nav` (1,0,0) over `.header` (0,1,0). The `:where(:hover)` pseudo-class resets its specificity to 0, so only `.btn` contributes (0,1,0). The descendant combinator sums these, resulting in an overall specificity of (1,1,0).

Using `:is()` and `:where()` in this way optimizes specificity by promoting intentional weighting and reducing accidental selector bloat, leading to more scalable and maintainable CSS.

When using `@layer`, the styles are applied in the order of the layers, with later layers overriding earlier ones. However, within each layer, the standard CSS specificity rules still apply.

Jane also discussed performance and specificity with the interviewer. While high-specificity selectors—such as chained classes or IDs—typically have